

```

---
title: "algoritmos"
output:
  html_document: default
  word_document: default
  pdf_document: default
author: "Leslie Garcia"
date: "2026-04-01"
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

## R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF,
and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the Knit button a document will be generated that includes both content as
well as the output of any embedded R code chunks within the document. You can embed an R code
chunk like this:
##HECHO EN CLASE
## Primeros comandos en R

R es muy parecido a matlab y permite el trabajo con matrices
```{r}
A<-38
A<-40
B<-60
C<-B-A
C
```

```{r cars}
summary(cars)
```

## uso de datasets o base de datos internos de R

Usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos
de R, estos datos estan dado en la tabla de las cuales podemos elegir columnas específicas si
escribimos $ despues de el comando de informacion especifico

## Including Plots

You can also embed plots, for example:

```{r pressure, echo=FALSE}
plot(pressure) ##poner lo de la foto
```

## funciones estadísticas

```{r}
z1<-rnorm(350,22,5)
z1
```

```{r}
plot(z1)

w1<-length(z1)
w1
x1<-(500:849)
x1

```

```
plot(z1,x1)
```

```

```
```{r}
hist(z1, main="Histograma de z1", col="lightblue")
```

```

```
```{r}
density(z1)
```

```

```
```{r}
plot(density(z1))
Pedir lo de aca
```

```

Ejercicio 1 de algoritmos

Consigna: Las ultimas 3 cifras de mi DNI son 126, consigna crear una variable que tenga ese número

```
```{r}
DNI=126
```

```

consigna: crear un vector que tenga todos los números desde el 1 hasta el 126

```
```{r}
SECUENCIADNI<-(1:126)
SECUENCIADNI
```

```

Ejercicio 2

Calcular la suma de todos los valores del vector SECUENCIADNI

```
```{r}
TOTAL<-0
VALORFINAL<-length(SECUENCIADNI)
for (i in 1:VALORFINAL){
 TOTAL<-TOTAL+i }
TOTAL
```

```

##Ejercicio 3

##Repetir el ejercicio anterior pero en Python

```
```{python}
suma=0
for i in range (1,127):
 # range llega hasta uno antes del final, por eso 127
 suma+=i # <--- Esta línea DEBE tener un espacio al principio
print(suma)
```

```

##Ejercicio 4

##Hacer lo mismo en Julia

no funciona

hacer las consignas del pdf en github del profe, en industrial

##HASTA ACÁ LLEGA LO HECHO EN CLASE

1.4 Generar un vector secuencia

```
```{r}
Método con for
A <- 0
start_for <- Sys.time()
for (i in 1:50000) { A[i] <- (i*2) }
end_for <- Sys.time()
tiempo_for <- end_for - start_for
```

```

```

# Método con seq() de R
start_seq <- Sys.time()
B <- seq(2, 100000, by = 2)
end_seq <- Sys.time()
tiempo_seq <- end_seq - start_seq

# Mostrar resultados
head(A)
tail(A)
head(B)
tail(B)

# Comparar tiempos
print(paste('Tiempo for:', tiempo_for))
print(paste('Tiempo seq:', tiempo_seq))

...

1.5 Implementación de una serie Fibonacci o Fibonacci
```{r}
fibonacci <- function(n) {
 fib <- c(0, 1)
 while (tail(fib, 1) < n) {
 len <- length(fib)
 fib <- c(fib, fib[len] + fib[len - 1])
 }
 return(fib)
}

Generar hasta superar 1.000.000
resultado <- fibonacci(1000000)
print(resultado)
print(paste('Iteraciones necesarias:', length(resultado)))
print(paste('Ultimo numero mayor a 1.000.000:', tail(resultado, 1)))

...

1.7 Ordenación de un vector por método burbuja
```{r}
# Cargamos la librería para medir performance
library(microbenchmark)

# 1. Creamos la muestra de tamaño 20.000
x <- sample(1:100000, 20000)

# 2. Definimos la función de la burbuja
burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n - 1)) {
    for (i in 1:(n - j)) {
      if (x[i] > x[i + 1]) {
        temp <- x[i]
        x[i] <- x[i + 1]
        x[i + 1] <- temp
      }
    }
  }
  return(x)
}

# 3. Comparamos la performance
comparacion <- microbenchmark(
  Metodo_Burbuja = burbuja(x),
  Metodo_Sort_R = sort(x),
  times = 1
)

# 4. Mostramos el resultado
print(comparacion)
```

```

## 1.8 La penitencia de Newton

```

```{r}
# Metodo 1: suma con for
metodo_for <- function() {
  numeros <- 1:1e10
  suma <- 0
  for (i in numeros) {
    suma <- suma + i
  }
  return(suma)
}

# Metodo 2: formula matematica de Gauss
metodo_formula <- function() {
  n <- 1e10
  suma <- n * (n + 1) / 2
  return(suma)
}

# Medir tiempo metodo for (advertencia: puede tardar mucho)
# Para prueba rapida usar 1:1e6 en lugar de 1:1e10
tiempo_for <- system.time({
  resultado_for <- metodo_for()
})

# Medir tiempo metodo formula
tiempo_formula <- system.time({
  resultado_formula <- metodo_formula()
})

# Mostrar resultados
print(resultado_for)
print(resultado_formula)

# Mostrar tiempos
print(tiempo_for)
print(tiempo_formula)
```

```

El algoritmo Bubble Sort (Burbuja) tiene una complejidad computacional de  $O(n^2)$ . Esto significa que si duplicamos la cantidad de datos, el tiempo de procesamiento se cuadruplica. Es un método "pesado" porque realiza múltiples pasadas e intercambios manuales. Por otro lado, la función `sort()` de R utiliza algoritmos avanzados (como Quicksort o Shellsort) con una complejidad de  $O$  elevado a la 2, además de estar implementada en lenguaje C, lo que la hace órdenes de magnitud más eficiente.

```

```{r}
library(microbenchmark)
library(ggplot2)

# Implementación simple de bubble sort
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (x[j] > x[j+1]) {
        temp <- x[j]
        x[j] <- x[j+1]
        x[j+1] <- temp
      }
    }
  }
  return(x)
}

# Datos de prueba
set.seed(123)
vec <- sample(1:1000, 100)

```

```
# Benchmark
bench <- microbenchmark(
  bubble = bubble_sort(vec),
  sortR = sort(vec),
  times = 50
)
```

```
# Gráfico de violín
autoplot(bench)
```
```

El K-means es un algoritmo de particionamiento de datos (Clustering) que agrupa observaciones en k grupos basándose en la distancia a un centroide. Es un proceso iterativo que:

1. Define k centros iniciales.
2. Asigna cada punto al centroide más cercano (distancia euclidiana).
3. Recalcula el centroide como el promedio de los puntos asignados. Este ciclo se repite hasta que las asignaciones no cambian o se alcanza un límite de iteraciones. Es ampliamente usado en ingeniería para segmentación de mercado, detección de anomalías y compresión de datos.

```
```{r}
library(microbenchmark)
library(ggplot2)

set.seed(123)
data <- matrix(rnorm(1000), ncol=2)

bench_kmeans <- microbenchmark(
  kmeans2 = kmeans(data, centers=2),
  kmeans5 = kmeans(data, centers=5),
  times = 50
)

autoplot(bench_kmeans)
```
```